

Recitation 05

While Repetition Structure

Question 1 | Predict output

Predict the printed output obtained when running the following programs.

Program 1	Output 1
<pre> a = "python" b = "" i = len(a) = 6 while i > 0: b = b + str(i) + ":" + '{:->{w}}'.format(a[:i], w=len(a)) + "\n" i -= 1 print(b + "0:" + "-" * len(a)) </pre> <i>Handwritten notes:</i> 向右对齐 w=len(a)=6 长度对齐, format 格式, 打印换行	<pre> 6:python 5:-pytho 4:--pyth 3:---pyt 2:----py 1:-----p 0:----- </pre>
Program 2	Output 2
<pre> m = 6 n = 3 i = 1 while i < m: j = 1 while j <= i and i <= n: print('*', end=' ') j += 1 j += 1 while j <= m and i > n: print('*', end=' ') j += 1 print() i += 1 </pre> <i>Handwritten notes:</i> 换行, end="\n", m=3 时 i=n, 第一个循环不执行, 所以对齐 *	<pre> * * * * * * * * * * * * * * * </pre>
Program 3	Output 3
<pre> a = 0 b = 1 while a < 10: print(a) c, a = a, b b = c + b </pre> <i>Handwritten notes:</i> 斐波那契数列, 注意设置, 默认 end="\n" sep="0", a=0→1→1→2→3→5→8→13, b=1→1→2→3→5→8→13→21, c=→0→1→1→2→3→5→8	<pre> 0 1 1 2 3 5 8 </pre>

A-65 a-97 不要搞反

* x, s 没有重新赋值, 不会写值

以上一个结果的 s 字符串换行连接

Question 2 | Display pattern

Consider the following program:

```
m = 5
n = 3
x = ord('A') = 65
s = ""
i = 1

while i < m:
    j = 1
    while j <= i and i <= n:
        s += chr(x) + '|'
        j += 1
        x += 1
    j += i
    while j <= m and not i > n:
        s += chr(x) + '|'
        j += 1
        x += 1
    print(s)
    i += 1
```

看清楚 i

m=5
n=3
x=65 → 66 → 67 → 68 → 69 → 70 → 71 → 72 → 73 → 74 → 75
i=1 → 2 → 3 → 4 → 5
~~j=1 → 2 → 3 → 4 → 5 → 6~~
~~j=1 → 2 → 3 → 4 → 5 → 6~~
~~j=1 → 2 → 3 → 4 → 5~~
~~j=1 → 5~~

What is the printed output? Draw a grid you will use as a guide; each individual character (including blank spaces) of output should be placed in a single box.

A		B		C		D					
A		B		C		D		E		F	
A		B		C		D		E		F	
A		B		C		D		E		F	

```
A | B | C | D |
A | B | C | D | E | F | G |
A | B | C | D | E | F | G | H | I | J |
A | B | C | D | E | F | G | H | I | J |
```

Question 3 | Beautiful numbers

- 1) The following numbers are examples of narcissistic

$$371 = 3^3 + 7^3 + 1^3$$

$$9474 = 9^4 + 4^4 + 7^4 + 4^4$$

A narcissistic number is an n -digit number equal to the sum of its digits raised to the n^{th} power.

Write a program that prompts the user for a positive integer number and check if this number is a narcissistic number.

```
n = input("Enter a number:")
sum = 0
i = 1
while i <= len(n):
    sep = int(n[i-1]) ** len(n)
    sum += sep
    i += 1
if sum == int(n):
    print(f"{n} is a narcissistic number.")
else:
    print(f"{n} isn't a narcissistic number.")
```

- 2) The following numbers are examples of Kaprekar numbers:

$$9^2 = 81,$$

$$8 + 1 = 9;$$

$$45^2 = 2025,$$

$$20 + 25 = 45;$$

$$297^2 = 88209,$$

$$88 + 209 = 297;$$

$$4879^2 = 23804641,$$

$$238 + 4641 = 4879;$$

$$17344^2 = 300814336,$$

$$3008 + 14336 = 17344;$$

$$538461^2 = 289940248521, \quad 289940 + 248521 = 538461.$$

```
n = int(input("Enter a number:"))
square_n = n ** 2
if len(str(square_n)) % 2 == 0:
    sep_1 = int(str(square_n)[0:len(str(n)):1])
    sep_2 = int(str(square_n)[len(str(n)):])
    if sep_1 + sep_2 == n:
        print(f"{n} is a Kaprekar number.")
    else:
        print(f"{n} isn't a Kaprekar number.")
else:
    sep_3 = int(str(square_n)[0:len(str(n))-1:1])
    sep_4 = int(str(square_n)[len(str(n))-1:])
    if sep_3 + sep_4 == n:
        print(f"{n} is a Kaprekar number.")
    else:
        print(f"{n} isn't a Kaprekar number.")
```

To check if a positive whole number n that has d number of digits, check the following:

- Take the square n and separate the result into two pieces: a right-hand piece that has d digits and a left-hand piece that has either d or $d-1$ digits;
- Add these two pieces together.

If the result is n , then n is a Kaprekar number.

Question 4 | String manipulation

- 1) Give in one sentence what the goal of the following program

```
N = input('Enter a string: ')
```

```
i, j = 0, 0
while i < len(N):
    j = i + 1
    while j < len(N) and N[j] == N[i]:
        j = j + 1
    print(N[i], end='')
    i = j
```

Remove duplicated numbers and reserve the first one

deduplication

去重

检查相邻两次是否有重复

```
= RESTART: C:/Users/nolan/Desktop/NYU S.
/Python(ICP)/recitation 5(2).py
Enter a string: abcdeabcdeabcde
abcdeabcdeabcde
```

```
= RESTART: C:/Users/nolan/Desktop/NYU S.
/Python(ICP)/recitation 5(2).py
Enter a string: aabbccddeeff
abcdef
```

```
= RESTART: C:/Users/nolan/Desktop/NYU S.
/Python(ICP)/recitation 5(2).py
Enter a string: abcaabbccaaabbbcccd
abcabcabcd
```

若有保留其一，实现去重

Predict what the output will be, given for the input(s) listed in the table. Give your answer

Input	Expected output
abcdeabcdeabcde	abcdeabcdeabcde (一样)
aabbccddeeff	abcdef
abcaabbccaaabbbcccd	abcabcabcd

in the last column of this table.

- 2) Write a program that outputs the number of times a string appear into another.

Example output:

```
>>>Write a string: abcdeabcdeabcde
>>>Write another string: abc
String 'abc' appears 3 times.
```

```
string_1 = input("Write a string:")
string_2 = input("Write another string:")
total = 0
i = 0
while i <= len(string_1) - 3:
    if string_2 == string_1[i:i+3:1]:
        total += 1
    i += 1
print(f"String {string_2} appears {total} times.")
```

Question 5 | ISBN checksum

An ISBN (International standard book number) is a number that uniquely identifies commercial books. Before 2007, the ISBNs were 10-digit long and after that year, ISBNs were extended to 13 digits.

In both ISBN-10 and ISBN-13, the last digit of an ISBN is called the *check digit*. The check digit is used to detect erroneous ISBNs:

- In ISBN-10, the check digit can be an integer between 0 and 9 or letter 'X'.
- In ISBN-13, the check digit can be an integer between 0 and 9.

The purpose of this question is to write a program that checks if a number is a valid ISBN-10 or a valid ISBN-13.

For **ISBN-10** numbers, the check digit is computed as follows:

1. First, calculate s , the weighted sum of the first 9 digits with decreasing weights, the weight of the first left digit being 10.
2. Then, calculate n , the number needed to add to s to make it dividable by 11.
3. Finally, if n is equal to 10, the check digit is letter 'X'. Otherwise, the check digit is n .

Example: 0306406152

- weighted sum: $0 \times 10 + 3 \times 9 + 0 \times 8 + 6 \times 7 + 4 \times 6 + 0 \times 5 + 6 \times 4 + 1 \times 3 + 5 \times 2 = 130$
- check digit: $(130 + 2) \% 11 = 0$
- 0306406152 is a valid ISBN-10

For **ISBN-13** numbers, the check digit is computed as follows:

1. First calculate s , the weighted sum of the first 12 digits with alternate weights of 1 and 3, the weight of the first left digit being 1;
2. Then, calculate n , the number needed to add to s to make it dividable by 10.
3. The check digit is n .

Example: 9780306406157

- weighted sum: $9 \times 1 + 7 \times 3 + 8 \times 1 + 0 \times 3 + 3 \times 1 + 0 \times 3 + 6 \times 1 + 4 \times 3 + 0 \times 1 + 6 \times 3 + 1 \times 1 + 5 \times 3 = 93$
- check digit: $(93 + 7) \% 10 = 0$
- 9780306406157 is a valid ISBN-13

Program requirements

- Your program should prompt the user for a number with message "Enter an ISBN number:"
- Your program should check if the number of digits entered are 10 or 13. If not, the output should be "Not a valid ISBN."
- If the number of digits entered is 10 (respectively 13), your program should compute the check digit based on the 9 first digits (respectively the first 12 digits).
- if the check number computed by your program corresponds to the last digit, the output should be "Valid ISBN-10" or "Valid ISBN-13" depending on the number of digits first entered.
- if the check bit computed by your program does no correspond to the last digit entered, the output should be "Not a valid ISBN-10" or "Valid ISBN-13".

Sample output:

```
>>> Enter an ISBN number: 020151141
>>> Not a valid ISBN

>>> Enter an ISBN number: 020151141X
>>> Valid ISBN-10

>>> Enter an ISBN number: 0306406153
>>> Not a valid ISBN-10

>>> Enter an ISBN number: 0306406152
>>> Valid ISBN-10

>>> Enter an ISBN number: 9782340076556
>>> Valid ISBN-13
```

```
n = input("Enter an ISBN number:")
l = len(n)
i = 0
even_i = 0
odd_i = 0
weighted_sum_10 = 0
if l != 10 and l != 13:
    print("Not a valid ISBN")
elif l == 13:
    while i < l-1:
        if i % 2 == 0:
            even_i += int(n[i])
        if i % 2 == 1:
            odd_i += int(n[i]) * 3
        weighted_sum_13 = even_i + odd_i
        i += 1
    if (weighted_sum_13 + int(n[l-1])) % 10 == 0:
        print("Valid ISBN-13.")
    else:
        print("Not a valid-13.")
elif l == 10:
    while i < l-1:
        weighted_sum_10 += int(n[i]) * (10-i)
        i += 1
    if n[l-1] != 'X':
        if (weighted_sum_10 + int(n[l-1])) % 11 == 0:
            print("Valid ISBN-10.")
        else:
            print("Not a valid ISBN-10.")
    elif n[l-1] == 'X':
        if (weighted_sum_10 + 10) % 11 == 0:
            print("Valid ISBN-10.")
        else:
            print("Not a valid ISBN-10.")
```